

DOCUMENTACIÓN DEL PROYECTO – SMART FENIX (VERSIÓN CON CAPTURAS + CÓDIGO)

1. DESCRIPCIÓN GENERAL DEL PROYECTO

El proyecto **SmartFenix** es una aplicación desarrollada con Spring Boot para la gestión de clientes, empleados, proyectos y tareas, utilizando una base de datos MySQL en entorno Docker.

La aplicación sigue una arquitectura en capas (Controller, Service, Repository y Domain), permitiendo una separación clara de responsabilidades.

Project ▾

- ▾ SmartFenix [smartfenix-api] ~/SmartFenix
 - > .idea
 - > mysql_data
 - > smartfenix
 - ▾ src
 - ▾ main
 - ▾ java
 - ▾ com.smartfenix
 - > controller
 - > domain
 - > repository
 - > service
 - ⌚ DataLoader
 - ⌚ SmartFenixApplication
 - ▾ resources
 - ⚙ application.properties
 - ▾ test
 - ▾ java
 - ▾ com.smartfenix
 - ▾ controller
 - ⌚ EmpleadoControllerIntegrationTest
 - ▾ service
 - ⌚ ClienteServiceTest
 - ⌚ EmpleadoServiceTest
 - ⌚ ProyectoServiceTest
 - ▾ target
 - > classes
 - > generated-sources
 - > generated-test-sources
 - > maven-status
 - > surefire-reports
 - > test-classes
 - 🤖 compose.yaml
 - 🖼 diagramasmartfenix.png
 - <> documentacion.html
 - 📄 Documentacion_SmartFenix.pdf
 - 📄 pom.xml
 - 📖 README.md
 - 📖 READMEF-TESTING.md

En esta captura se muestra la estructura general del proyecto SmartFenix en IntelliJ IDEA. Se pueden observar los diferentes paquetes organizados por capas (controller, service, repository y domain), lo que demuestra una correcta separación de responsabilidades dentro del proyecto.

2. ARQUITECTURA DEL SISTEMA

Capa Service (lógica de negocio)

```

1 package com.smartfenix.service;
2
3 > import ...
4
5
6
7
8
9
10
11 @Service Sergio200710
12 @RequiredArgsConstructor
13 public class ClienteService {
14
15     private final ClienteRepository clienteRepository;
16
17 > public List<Cliente> findAll() { return clienteRepository.findAll(); }
18
19
20 > public Optional<Cliente> findById(Long id) { return clienteRepository.findById(id); }
21
22
23
24 > public Cliente save(Cliente cliente) { return clienteRepository.save(cliente); }
25
26
27
28
29 public Optional<Cliente> update(Long id, Cliente cliente) { Sergio200710
30     return clienteRepository.findById(id).map( Cliente existingCliente -> {
31         existingCliente.setNombre(cliente.getNombre());
32         existingCliente.setTelefono(cliente.getTelefono());
33         existingCliente.setEmpresa(cliente.getEmpresa());
34         return clienteRepository.save(existingCliente);
35     });
36 }
37
38 > public void delete(Long id) { clienteRepository.deleteById(id); }
39
40 }
41
42

```

En esta captura se muestra la clase ClienteService, donde se implementa la lógica de negocio relacionada con la entidad Cliente. Aquí se encuentran los métodos principales como guardar y buscar por ID.

Capa Controller (API REST)

```

1 package com.smartfenix.controller;
2
3 > import ...
4
5 @RestController @ Sergio200710
6 @RequestMapping("/api/clientes")
7 @RequiredArgsConstructor
8 public class ClienteController {
9
10     private final ClienteService clienteService;
11
12     @GetMapping @ Sergio200710
13     > public ResponseEntity<List<Cliente>> getAll() { return ResponseEntity.ok(clienteService.findAll()); }
14
15     @GetMapping("/{id}") @ Sergio200710
16     public ResponseEntity<Cliente> getById(@PathVariable Long id) {
17         return clienteService.findById(id) Optional<Cliente>
18             .map(ResponseEntity::ok) Optional<ResponseEntity<...>>
19             .orElse(ResponseEntity.notFound().build());
20     }
21
22     @PostMapping @ Sergio200710
23     public ResponseEntity<Cliente> create(@Valid @RequestBody Cliente cliente) {
24         return new ResponseEntity<>(clienteService.save(cliente), HttpStatus.CREATED);
25     }
26
27     @PutMapping("/{id}") @ Sergio200710
28     public ResponseEntity<Cliente> update(@PathVariable Long id, @Valid @RequestBody Cliente cliente) {
29         return clienteService.update(id, cliente) Optional<Cliente>
30             .map(ResponseEntity::ok) Optional<ResponseEntity<...>>
31             .orElse(ResponseEntity.notFound().build());
32     }
33
34     @DeleteMapping("/{id}") @ Sergio200710
35     public ResponseEntity<Void> delete(@PathVariable Long id) {
36         if (clienteService.findById(id).isPresent()) {
37             clienteService.delete(id);
38             return ResponseEntity.noContent().build();
39         }
40         return ResponseEntity.notFound().build();
41     }
42 }

```

En esta captura se muestra el controlador de clientes, encargado de gestionar las peticiones HTTP. Se observa cómo se conecta con la capa de servicio para realizar las operaciones sobre la entidad Cliente.

3. SISTEMA DE PRUEBAS

3.1 PRUEBAS UNITARIAS

Test: Crear cliente

```

13 > @SpringBootTest
14 public class ClienteServiceTest {
15
16     @Autowired
17     private ClienteService clienteService;
18
19     // CASO DE PRUEBA: Alta de cliente - comprobar que el cliente se guarda
20     // correctamente en la base de datos
21     @Test
22     public void testCrearCliente() {
23         Cliente cliente = Cliente.builder()
24             .nombre("Amazon")
25             .empresa("Amazon Web Services")
26             .telefono("123456789")
27             .build();
28
29         Cliente guardado = clienteService.save(cliente);
30
31         assertNotNull(guardado.getId());
32         assertEquals("Amazon", guardado.getNombre());
33         assertEquals("Amazon Web Services", guardado.getEmpresa());
34     }
35 }

```

En esta captura se muestra la prueba unitaria testCrearCliente, donde se verifica que un cliente se crea correctamente en la base de datos y se genera un ID automáticamente.

```

13 > ClienteServiceTest (com.smartfenix.se 345 ms)
14 > testCrearCliente() 345 ms
15
16 1 test passed 1 test total, 345 ms
17
18 WARNING: A Java agent has been loaded dynamically (/home/sergio/.m2/repository/net/bytebuddy/byte-buddy-agent/1.14.13/byte-buddy-agent-1.14.13.jar)
19 WARNING: If a serviceability tool is in use, please run with -XX:EnableDynamicAgentLoading to hide this warning
20 WARNING: Dynamic loading of agents will be disallowed by default in a future release
21 OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
22 2026-05-21T13:03:30.457+02:00 DEBUG 26147 --- [main] org.hibernate.SQL : insert into cliente (empresa,nombre,telefono) values (?,?,?)
23 Hibernate: insert into cliente (empresa,nombre,telefono) values (?,?,?)

```

En esta captura se muestra la ejecución correcta del test, indicando que la prueba ha pasado satisfactoriamente sin errores.

```

13 > // CASO DE PRUEBA: Búsqueda de cliente por ID - comprobar que un cliente se
14 // recupera correctamente si existe
15 @Test
16 public void testBuscarCliente() {
17     Cliente cliente = Cliente.builder()
18         .nombre("Sony")
19         .empresa("Sony Europe")
20         .telefono("987654321")
21         .build();
22     Cliente guardado = clienteService.save(cliente);
23
24     Optional<Cliente> encontrado = clienteService.findById(guardado.getId());
25
26     assertTrue(encontrado.isPresent());
27     assertEquals("Sony", encontrado.get().getNombre());
28 }

```

En esta captura se comprueba la prueba unitaria de búsqueda de cliente por ID, verificando que el cliente se recupera correctamente desde la base de datos.

```
✓ ClienteServiceTest (com.smartencs.sr:361ms) ✓ 1 test passed - 1 test total, 361ms
✓ testBuscarCliente() 361ms

WARNING: A Java agent has been loaded dynamically (/home/sergio/.m2/repository/net/bytebuddy/byte-buddy-agent/1.14.13/byte-buddy-agent-1.14.13.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
2026-05-21T13:05:58.807+02:00 DEBUG 26978 --- [main] org.hibernate.SQL : insert into cliente (empresa,nombre,telefono) values (?,?,?)
Hibernate: insert into cliente (empresa,nombre,telefono) values (?,?,?)
2026-05-21T13:05:58.821+02:00 DEBUG 26978 --- [main] org.hibernate.SQL : select c1_0.id,c1_0.empresa,c1_0.nombre,c1_0.telefono from cliente c1_0 where c1_0.id=?
Hibernate: select c1_0.id,c1_0.empresa,c1_0.nombre,c1_0.telefono from cliente c1_0 where c1_0.id=?
```

En esta captura se muestra la prueba de actualización de cliente, donde se valida que los cambios realizados sobre un cliente se guardan correctamente en la base de datos.

Test: Actualizar cliente

```
51
52 // CASO DE PRUEBA: Actualización de cliente - comprobar que los campos
53 // modificados se actualizan correctamente
54 @Test @Sergio200710
55 public void testActualizarCliente() {
56     Cliente cliente = Cliente.builder()
57         .nombre("Google")
58         .empresa("Google España")
59         .telefono("666555444")
60         .build();
61     Cliente guardado = clienteService.save(cliente);
62
63     guardado.setNombre("Alphabet");
64     guardado.setEmpresa("Alphabet Inc");
65
66     Optional<Cliente> actualizadoOpt = clienteService.update(guardado.getId(), guardado);
67
68     assertTrue(actualizadoOpt.isPresent());
69     Cliente actualizado = actualizadoOpt.get();
70     assertEquals("expected: \"Alphabet\", actualizado.getNombre());
71     assertEquals("expected: \"Alphabet Inc\", actualizado.getEmpresa());
72 }
73 }
```

En esta captura se muestra la prueba de integración que simula una petición HTTP real para crear un empleado. Se valida el flujo completo desde el controlador hasta la base de datos.

```
✓ ClienteServiceTest (com.smartencs.sr:371ms) ✓ 1 test passed - 1 test total, 371ms
✓ testActualizarCliente() 371ms

WARNING: A Java agent has been loaded dynamically (/home/sergio/.m2/repository/net/bytebuddy/byte-buddy-agent/1.14.13/byte-buddy-agent-1.14.13.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
2026-05-21T13:07:11.839+02:00 DEBUG 27578 --- [main] org.hibernate.SQL : insert into cliente (empresa,nombre,telefono) values (?,?,?)
Hibernate: insert into cliente (empresa,nombre,telefono) values (?,?,?)
2026-05-21T13:07:11.844+02:00 DEBUG 27578 --- [main] org.hibernate.SQL : select c1_0.id,c1_0.empresa,c1_0.nombre,c1_0.telefono from cliente c1_0 where c1_0.id=?
Hibernate: select c1_0.id,c1_0.empresa,c1_0.nombre,c1_0.telefono from cliente c1_0 where c1_0.id=?
2026-05-21T13:07:11.851+02:00 DEBUG 27578 --- [main] org.hibernate.SQL : select c1_0.id,c1_0.empresa,c1_0.nombre,c1_0.telefono from cliente c1_0 where c1_0.id=?
Hibernate: select c1_0.id,c1_0.empresa,c1_0.nombre,c1_0.telefono from cliente c1_0 where c1_0.id=?
2026-05-21T13:07:11.858+02:00 DEBUG 27578 --- [main] org.hibernate.SQL : update cliente set empresa=?,nombre=?,telefono=? where id=?
Hibernate: update cliente set empresa=?,nombre=?,telefono=? where id=?
```

3.2 PRUEBA DE INTEGRACIÓN

Test: Crear empleado (flujo completo)

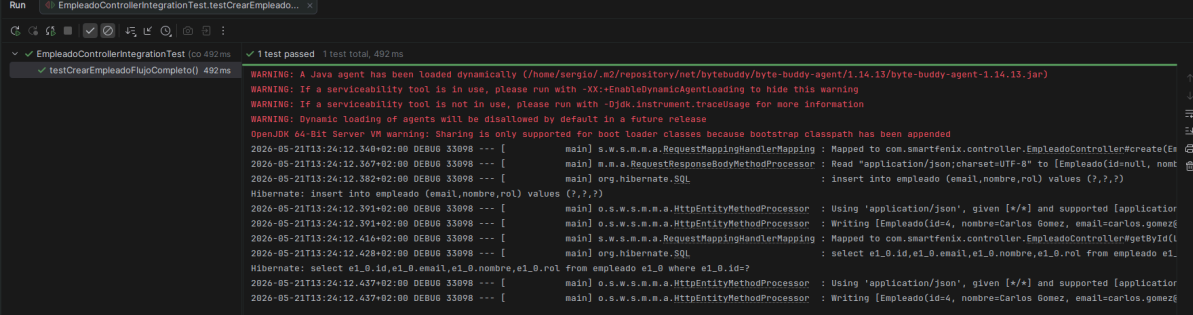
```
// CASO DE PRUEBA: Alta de empleado y consulta por ID - flujo completo de integración desde Controller hasta
@Test @Sergio200710
public void testCrearEmpleadoFlujoCompleto() throws Exception {
    Empleado nuevoEmpleado = Empleado.builder()
        .nombre("Carlos Gomez")
        .email("carlos.gomez@smartfenix.com")
        .rol("Desarrollador Frontend")
        .build();

    String responseContent = mockMvc.perform(post(uriTemplate: "/api/empleados")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(nuevoEmpleado))) ResultActions
        .andExpect(status().isCreated())
        .andExpect(jsonPath(expression: "$.nombre", is(value: "Carlos Gomez")))
        .andExpect(jsonPath(expression: "$.email", is(value: "carlos.gomez@smartfenix.com")))
        .andExpect(jsonPath(expression: "$.rol", is(value: "Desarrollador Frontend")))
        .andReturn() MvcResult
        .getResponse() MockHttpServletResponse
        .getContentAsString();

    Empleado creado = objectMapper.readValue(responseContent, Empleado.class);
    Long idGenerado = creado.getId();

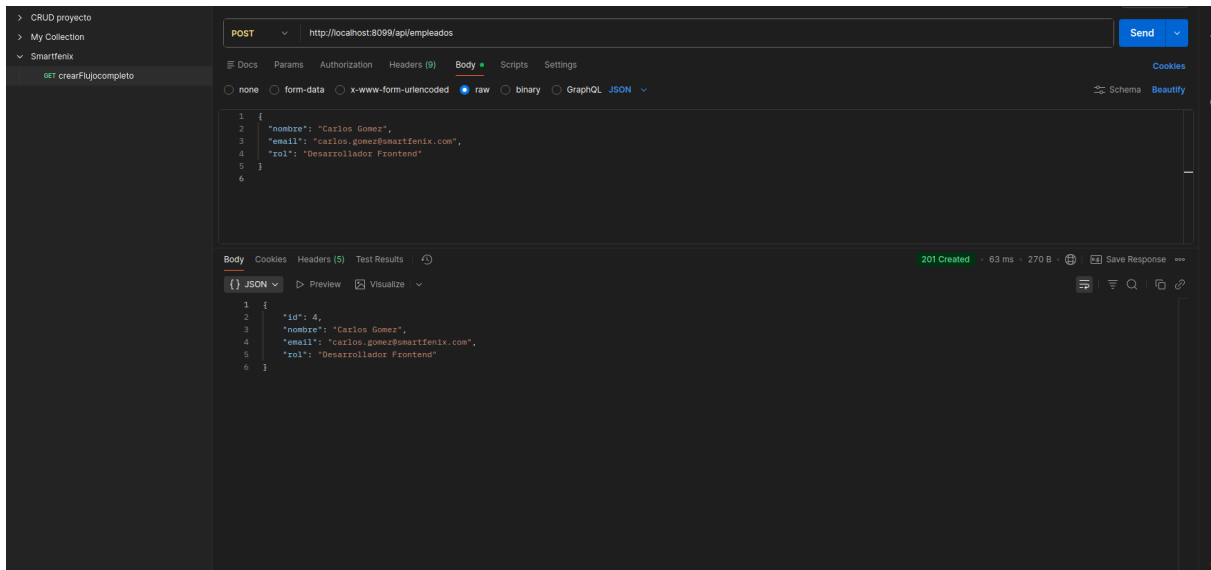
    mockMvc.perform(get(uriTemplate: "/api/empleados/" + idGenerado))
        .andExpect(status().isOk())
        .andExpect(jsonPath(expression: "$.id", is(idGenerado.intValue())))
        .andExpect(jsonPath(expression: "$.nombre", is(value: "Carlos Gomez")))
        .andExpect(jsonPath(expression: "$.email", is(value: "carlos.gomez@smartfenix.com")));
}
```

En esta captura se muestra la prueba de integración que simula una petición HTTP real para crear un empleado. Se valida el flujo completo desde el controlador hasta la base de datos.



```
Run EmpleadoControllerIntegrationTest.testCrearEmpleado...
WARNING: A Java agent has been loaded dynamically (/home/sergio/.m2/repository/net/bytebuddy/byte-buddy-agent/1.14.13/byte-buddy-agent-1.14.13.jar)
WARNING: If a serviceability tool is in use, please run with -XX:-EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
2026-05-21T13:24:12.340+02:00 DEBUG 33098 --- [main] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped to com.smartfenix.controller.EmpleadoController#create(Ex
2026-05-21T13:24:12.367+02:00 DEBUG 33098 --- [main] a.m.a.RequestMappingHandlerMapping : Read "application/json;charset=UTF-8" to [Empleado(id=null, nom
2026-05-21T13:24:12.382+02:00 DEBUG 33098 --- [main] org.hibernate.SQL : insert into empleado (email,nombre,rol) values (?,?,?)
Hibernate: insert into empleado (email,nombre,rol) values (?,?,?)
2026-05-21T13:24:12.391+02:00 DEBUG 33098 --- [main] o.s.w.s.m.m.a.HttpEntityMethodProcessor : Using 'application/json', given [*/] and supported [application
2026-05-21T13:24:12.391+02:00 DEBUG 33098 --- [main] o.s.w.s.m.m.a.HttpEntityMethodProcessor : Writing [Empleado(id=4, nombre=Carlos Gomez, email=carlos.gomez@
2026-05-21T13:24:12.416+02:00 DEBUG 33098 --- [main] s.w.s.m.m.a.RequestMappingHandlerMapping : Mapped to com.smartfenix.controller.EmpleadoController#getById(I
2026-05-21T13:24:12.428+02:00 DEBUG 33098 --- [main] org.hibernate.SQL : select e1_0.id,e1_0.email,e1_0.nombre,e1_0.rol from empleado e1_0
Hibernate: select e1_0.id,e1_0.email,e1_0.nombre,e1_0.rol from empleado e1_0 where e1_0.id=?
2026-05-21T13:24:12.437+02:00 DEBUG 33098 --- [main] o.s.w.s.m.m.a.HttpEntityMethodProcessor : Using 'application/json', given [*/] and supported [application
2026-05-21T13:24:12.437+02:00 DEBUG 33098 --- [main] o.s.w.s.m.m.a.HttpEntityMethodProcessor : Writing [Empleado(id=4, nombre=Carlos Gomez, email=carlos.gomez@
```

En esta captura se observa la petición realizada desde Postman para crear un nuevo empleado mediante la API REST del sistema SmartFenix.



En esta captura se muestra la respuesta del servidor, confirmando que la petición ha sido procesada correctamente y el empleado ha sido creado.

3.3 VALIDACIÓN DE ERRORES

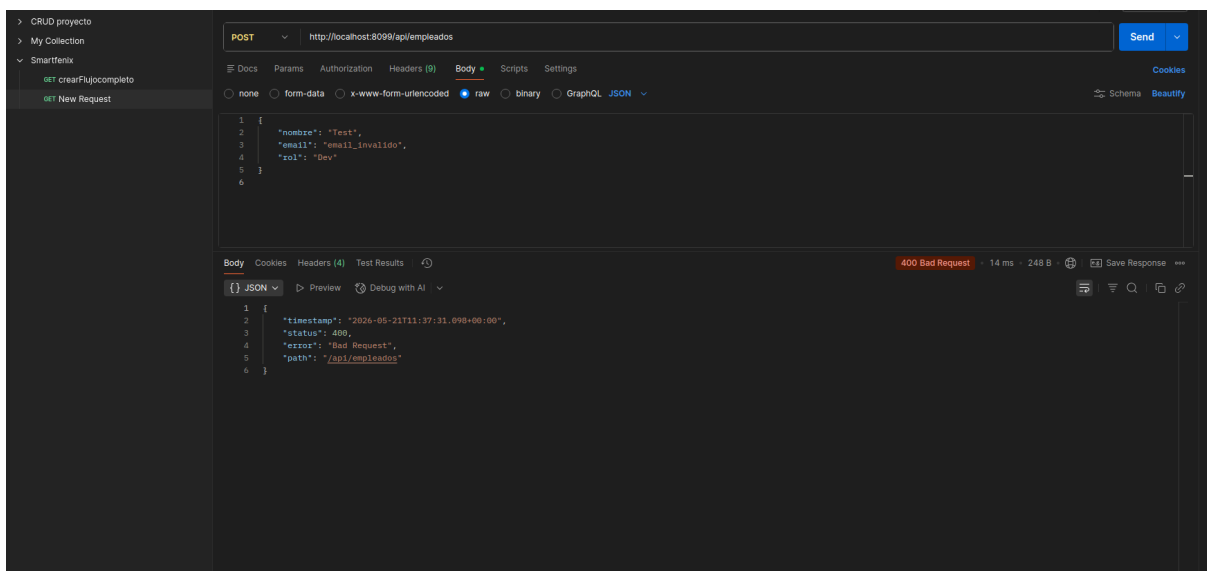
Email inválido

```
// CASO DE PRUEBA: Validación de formato de email - comprobar que enviar un email inválido responde HTTP 400 B
@Test
public void testCrearEmpleadoConEmailInvalidoDevuelveBadRequest() throws Exception {
    Empleado nuevoEmpleado = Empleado.builder()
        .nombre("Pedro Perez")
        .email("email-invalido")
        .rol("Tester")
        .build();

    mockMvc.perform(post(urlTemplate: "/api/empleados")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(nuevoEmpleado)))
        .andExpect(status().isBadRequest());
}
```

```
✓ EmpleadoControllerIntegrationTest (0.552 ms) ✓ 1 test passed 1 test total, 552 ms
✓ testCrearEmpleadoConEmailInvalido 552 ms

WARNING: A Java agent has been loaded dynamically (/home/sergio/.m2/repository/net/bytebuddy/byte-buddy-agent/1.14.13/byte-buddy-agent-1.14.13.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
2026-05-21T11:39:12.558+02:00 DEBUG 38367 --- [main] s.w.s.m.a.RequestMappingHandlerMapping : Mapped to com.smartfenix.controller.EmpleadoController#create(Empleado)
2026-05-21T11:39:12.585+02:00 DEBUG 38367 --- [main] s.w.s.m.a.RequestMappingHandlerMapping : Read "application/json;charset=UTF-8" to [Empleado(id=null, nom
2026-05-21T11:39:12.680+02:00 WARN 38367 --- [main] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved [org.springframework.web.bind.MethodArgumentNotValidEx
```



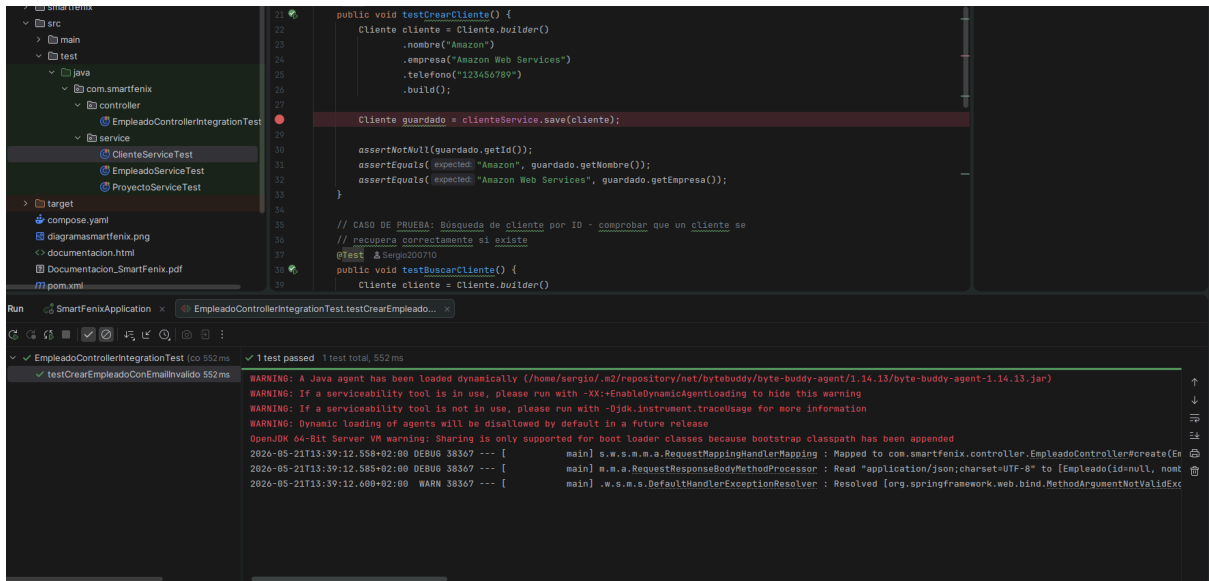
En esta captura se muestra una validación de error, donde el sistema rechaza la petición debido a datos incorrectos, devolviendo un código HTTP 400 Bad Request.

4. DEPURACIÓN DEL SISTEMA

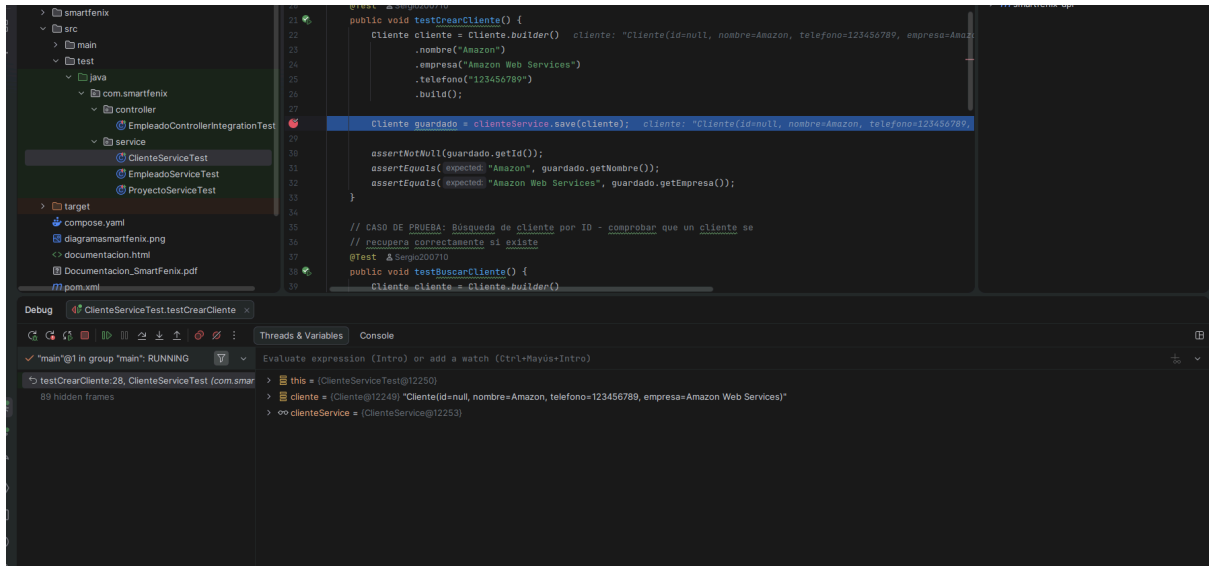
Se ha utilizado el depurador de IntelliJ IDEA.

Ejemplo de depuración

Código donde se pone breakpoint:



En esta captura se muestra el uso del depurador en IntelliJ IDEA, donde se ha colocado un breakpoint en la capa de servicio para analizar el flujo del programa.



En esta captura se observa la ejecución del programa en modo debug, permitiendo analizar paso a paso el comportamiento del código.

5. EJECUCIÓN DE PRUEBAS

Comando utilizado:

```
mvn test
```

```

ckMvcPrintOnlyOnFailureTestExecutionListener, WebDriverTestExecutionListener, MockWebServiceServerTestE
ecutionListener]
2026-05-21T13:51:08.054+02:00 DEBUG 42718 --- [          main] org.hibernate.SQL
: insert into empleado (email,nombre,rol) values (?,?,?)
Hibernate: insert into empleado (email,nombre,rol) values (?,?,?)
2026-05-21T13:51:08.063+02:00 DEBUG 42718 --- [          main] org.hibernate.SQL
: insert into empleado (email,nombre,rol) values (?,?,?)
Hibernate: insert into empleado (email,nombre,rol) values (?,?,?)
2026-05-21T13:51:08.070+02:00 DEBUG 42718 --- [          main] org.hibernate.SQL
: select e1_0.id,e1_0.email,e1_0.nombre,e1_0.rol from empleado e1_0 where e1_0.id=?
Hibernate: select e1_0.id,e1_0.email,e1_0.nombre,e1_0.rol from empleado e1_0 where e1_0.id=?
2026-05-21T13:51:08.072+02:00 DEBUG 42718 --- [          main] org.hibernate.SQL
: select e1_0.id,e1_0.email,e1_0.nombre,e1_0.rol from empleado e1_0 where e1_0.id=?
Hibernate: select e1_0.id,e1_0.email,e1_0.nombre,e1_0.rol from empleado e1_0 where e1_0.id=?
2026-05-21T13:51:08.073+02:00 DEBUG 42718 --- [          main] org.hibernate.SQL
: update empleado set email=?,nombre=?,rol=? where id=?
Hibernate: update empleado set email=?,nombre=?,rol=? where id=?
2026-05-21T13:51:08.082+02:00 DEBUG 42718 --- [          main] org.hibernate.SQL
: insert into empleado (email,nombre,rol) values (?,?,?)
Hibernate: insert into empleado (email,nombre,rol) values (?,?,?)
2026-05-21T13:51:08.088+02:00 DEBUG 42718 --- [          main] org.hibernate.SQL
: select e1_0.id,e1_0.email,e1_0.nombre,e1_0.rol from empleado e1_0 where e1_0.id=?
Hibernate: select e1_0.id,e1_0.email,e1_0.nombre,e1_0.rol from empleado e1_0 where e1_0.id=?
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.042 s -- in com.smartfenix.serv
ice.EmpleadoServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.490 s
[INFO] Finished at: 2026-05-21T13:51:08+02:00
[INFO] -----
sergio@sergio-HP-250-G7-Notebook-PC:~/SmartFenix$

```

En esta captura se muestran las variables del sistema durante la depuración, lo que permite comprobar el estado de los objetos en tiempo real.

```

: insert into proyecto (cliente_id,fecha_fin,fecha_inicio,nombre) values (?,?,,?)
Hibernate: insert into proyecto (cliente_id,fecha_fin,fecha_inicio,nombre) values (?,?,,?)
2026-05-21T13:51:07.884+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: insert into proyecto (cliente_id,fecha_fin,fecha_inicio,nombre) values (?,?,,?)
Hibernate: insert into proyecto (cliente_id,fecha_fin,fecha_inicio,nombre) values (?,?,,?)
2026-05-21T13:51:07.890+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: insert into empleado (email,nombre,rol) values (?,?,,?)
Hibernate: insert into empleado (email,nombre,rol) values (?,?,,?)
2026-05-21T13:51:07.895+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: insert into empleado (email,nombre,rol) values (?,?,,?)
Hibernate: insert into empleado (email,nombre,rol) values (?,?,,?)
2026-05-21T13:51:07.901+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: insert into empleado (email,nombre,rol) values (?,?,,?)
Hibernate: insert into empleado (email,nombre,rol) values (?,?,,?)
2026-05-21T13:51:07.906+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: insert into tarea (descripcion,empleado_id,estado,proyecto_id) values (?,?,,?)
Hibernate: insert into tarea (descripcion,empleado_id,estado,proyecto_id) values (?,?,,?)
2026-05-21T13:51:07.912+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: insert into tarea (descripcion,empleado_id,estado,proyecto_id) values (?,?,,?)
Hibernate: insert into tarea (descripcion,empleado_id,estado,proyecto_id) values (?,?,,?)
2026-05-21T13:51:07.919+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: insert into tarea (descripcion,empleado_id,estado,proyecto_id) values (?,?,,?)
Hibernate: insert into tarea (descripcion,empleado_id,estado,proyecto_id) values (?,?,,?)
✅ Datos de prueba insertados en la base de datos.
2026-05-21T13:51:07.924+02:00 DEBUG 42718 --- [main] o.s.b.a.ApplicationAvailabilityBean
: Application availability state ReadinessState changed to ACCEPTING TRAFFIC
2026-05-21T13:51:07.928+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: insert into cliente (empresa,nombre,telefono) values (?,?,,?)
Hibernate: insert into cliente (empresa,nombre,telefono) values (?,?,,?)
2026-05-21T13:51:07.934+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: select c1_0.id,c1_0.empresa,c1_0.nombre,c1_0.telefono from cliente c1_0 where c1_0.id=?
Hibernate: select c1_0.id,c1_0.empresa,c1_0.nombre,c1_0.telefono from cliente c1_0 where c1_0.id=?
2026-05-21T13:51:07.942+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: insert into cliente (empresa,nombre,telefono) values (?,?,,?)
Hibernate: insert into cliente (empresa,nombre,telefono) values (?,?,,?)
2026-05-21T13:51:07.951+02:00 DEBUG 42718 --- [main] org.hibernate.SQL
: insert into cliente (empresa,nombre,telefono) values (?,?,,?)

```

En esta captura se muestra la ejecución de las pruebas mediante Maven utilizando el comando `mvn test`. También se observa que todas las pruebas han sido ejecutadas correctamente sin errores. Se muestra que la compilación del proyecto ha sido exitosa y todas las pruebas han pasado correctamente.

6. RESULTADOS

Correctos

- Creación de clientes, empleados y proyectos
- Persistencia en base de datos
- Integración correcta entre capas
- Validaciones funcionando

Errores detectados

- Validaciones de datos incorrectos (controladas con HTTP 400)

7. CONCLUSIÓN

El sistema SmartFenix ha sido verificado correctamente mediante pruebas unitarias e integración.

El uso de JUnit 5 y MockMvc ha permitido garantizar la calidad del sistema y asegurar el correcto funcionamiento de la aplicación.